



TRANSFORMER

45 MOST IMPORTANT INTERVIEW QUESTIONS

— Master 45 Questions | Ace Your Interview —



Core Concepts

- Architecture
- Encoder / Decoder
- Positional Encoding
- Attention Mechanism



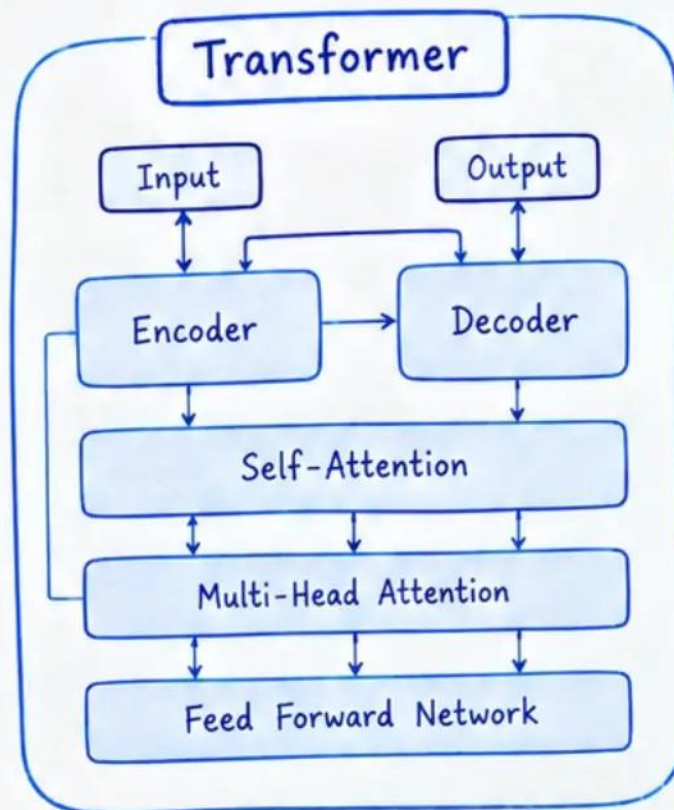
Algorithms & Math

- Scaled Dot-Product
- Multi-Head Attention
- Masking
- Positional Encoding



Implementation

- Model Components
- Hyperparameters
- Training Process
- Inference



From Sequences to Intelligence



Applications

- NLP (LLMs)
- Vision (ViT)
- Speech
- Multimodal



Advanced Topics

- Variants (BERT, GPT)
- Fine-tuning
- Optimization
- Limitations



Interview Focus

- Theory + Practical
- Comparisons
- Real-world Scenarios
- Common Mistakes



WHY TRANSFORMER IS IMPORTANT IN AI?

- ✓ Powers Modern LLMs
- ✓ Enables Better Context Understanding
- ✓ Improves Performance & Scalability
- ✓ Used in Text, Vision, Speech & Multimodal AI
- ✓ Shapes the Future of AI

Understand → Practice → Crack Your Interview

3 INTERVIEW QUESTIONS

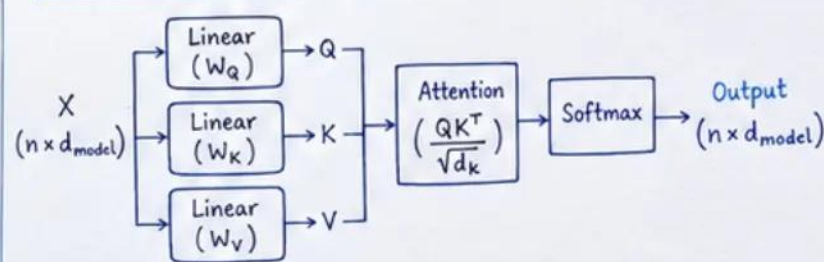
QUESTION 1 What is the intuition behind the self-attention mechanism in Transformers? How does it help in capturing long-range dependencies?

THEORETICAL SOLUTION:

- Self-attention allows each token to attend to all other tokens in the input sequence and compute a weighted representation.
- It uses **Query (Q)**, **Key (K)** and **Value (V)** vectors to calculate attention scores.
- The attention score between two tokens is based on the similarity of their Q and K vectors. If two tokens are similar, the model gives them more weight.
- This helps in capturing long-range dependencies because tokens far apart in the sequence can still have high similarity.
- Unlike RNNs/CNNs, self-attention is parallelizable and more efficient for long sequences.

★ INTERVIEW TIP:

Be clear about the role of Q, K, V and why scaling is required (to avoid large dot products).

Self-Attention (Single Head)Attention Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

★ IMPORTANT:

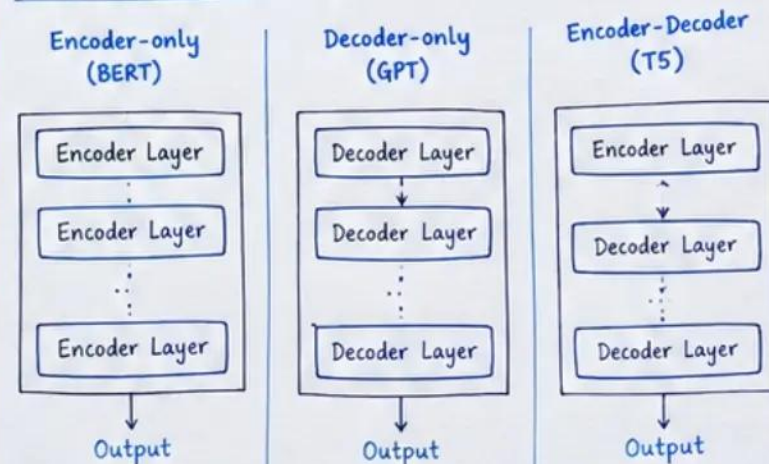
- Scaling by $\sqrt{d_k}$ prevents large dot products.
- Multi-head attention uses multiple heads to attend to information from different representation subspaces.

QUESTION 2 Explain the difference between encoder-only, decoder-only and encoder-decoder Transformer architectures. Give one example of each.

THEORETICAL SOLUTION:

- Encoder-only:** Used for understanding tasks (e.g., classification, sentence embedding). It only has an encoder stack.
- Decoder-only:** Used for generation tasks (e.g., language modeling). It only has a decoder stack with causal masking.
- Encoder-Decoder:** Used for sequence-to-sequence tasks (e.g., translation, summarization). It has both encoder and decoder stacks with cross-attention.

Architecture	Used for	Example
Encoder-only	Understanding	BERT
Decoder-only	Generation	GPT
Encoder-Decoder	Seq2Seq	T5, BART

Architecture Comparison:★ FOLLOW-UP:

Why does BERT use bidirectional attention while GPT uses causal masking?

QUESTION 3 What is positional encoding in Transformers? Discuss different types of positional encoding and why it is needed.

THEORETICAL SOLUTION:

- Positional encoding gives information about the position of tokens in the sequence, since self-attention is position agnostic.
- Without it, the model would not know the order of words.
- Types:**

1. Sinusoidal Positional Encoding (original paper)

- Uses sine and cosine function of different frequencies.
- Allows the model to attend by relative positions.

2. Learned Positional Encoding

- Position embeddings are learned as parameters.
- More flexible, works well in practice.

Sinusoidal PE:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

★ IMPORTANT:

- Positional encoding is added to input embeddings at the bottom of the model.
- Helps the model understand relative and absolute positions.

Comparison:

Type	Advantage	Disadvantage
Sinusoidal	Allows extrapolation to longer sequences	Fixed pattern, less flexible
Learned	More expressive, can learn patterns	No explicit position logic

★ INTERVIEW TIP:

Be ready to explain why positional encoding is needed and how it works mathematically.

3 INTERVIEW QUESTIONS

QUESTION 4 What is positional encoding in Transformers? Why is it needed and what are the different types of positional encoding?

THEORETICAL SOLUTION:

- Positional encoding is a way to give information about the position of tokens in a sequence. Since self-attention is position agnostic, the model needs some form of positional information.
- It helps the model understand the relative and absolute position of words, which is important for tasks like translation, summarization, etc.
- Without positional encoding, the model would treat all permutations of the same words the same.

TYPES:

- Sinusoidal Positional Encoding** (original paper)
 - Uses sine and cosine functions of different frequencies.
 - Allows the model to attend by relative positions.
- Learned Positional Encoding**
 - Position embeddings are learned as parameters.
 - More flexible, works well in practice.

Sinusoidal Positional Encoding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

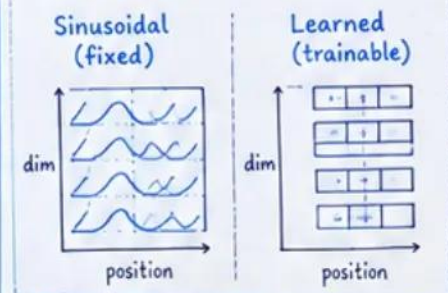
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Why needed?

- Gives sequence order information.
- Helps in capturing relative positions.
- Improves performance on sequence tasks.

★ INTERVIEW TIP:

Be ready to explain why positional encoding is added and how it works mathematically.

Types of Positional EncodingComparison:

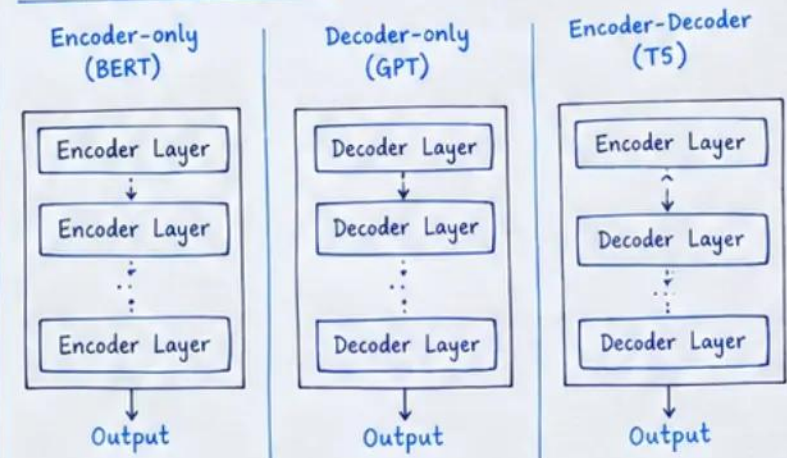
Feature	Sinusoidal	Learned
Nature	Fixed formula	Trainable
Flexibility	Less flexible	More flexible
Performance	Good	Often better in practice

QUESTION 5 Explain the difference between encoder-only, decoder-only and encoder-decoder Transformer architectures. Give one example of each.

THEORETICAL SOLUTION:

- Encoder-only:** Used for understanding tasks (e.g., classification, sentence embedding). It only has an encoder stack.
- Decoder-only:** Used for generation tasks (e.g., language modeling). It only has a decoder stack with causal masking.
- Encoder-Decoder:** Used for sequence-to-sequence tasks (e.g., translation, summarization). It has both encoder and decoder stacks with cross-attention.

Architecture	Used for	Example
Encoder-only	Understanding	BERT
Decoder-only	Generation	GPT
Encoder-Decoder	Seq2Seq	T5, BART

Architecture Comparison:★ FOLLOW-UP:

Why does BERT use bidirectional attention while GPT uses causal masking?

QUESTION 6 What are the common attention mechanisms used in Transformers? How do they differ and when are they used?

THEORETICAL SOLUTION:

- Self-Attention:** Attends to all positions in the same sequence. Used in encoder and decoder (masked in decoder).
- Cross-Attention:** Attends to encoder outputs. Used in decoder to focus on relevant source tokens.
- Multi-Head Attention:** Uses multiple attention heads to capture different types of relationships (syntactic, semantic, etc.).

★ IMPORTANT:

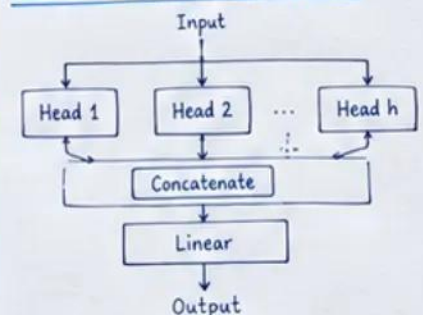
Multi-head attention allows the model to attend to information from different representation subspaces at different positions.

Types of Attention

Type	Query	Key/Value	Used in
Self-Attention	Same seq	Same seq	Encoder & Decoder (masked)
Cross-Attention	Decoder	Encoder	Decoder
Multi-Head	Multiple heads	Multiple heads	Encoder & Decoder

★ INTERVIEW TIP:

Be ready to explain the difference between self-attention, cross-attention and multi-head attention with a simple diagram.

Multi-Head Attention (simplified)When to use?

- Self-attention → Language modeling, classification
- Cross-attention → seq2seq, translation
- Multi-head → better representation learning

3 INTERVIEW QUESTIONS

QUESTION 7 What is the role of positional encoding in Transformers? Why is it necessary? What are the different types of positional encoding?

THEORETICAL SOLUTION:

- Positional encoding provides information about the position of tokens in the sequence. Since self-attention is position agnostic, the model needs position information to understand the order of words.
- It is necessary because without it, the model would treat all positions the same, leading to incorrect understanding of the sequence.
- Types of positional encoding:
 - Sinusoidal (fixed) – uses sine and cosine functions.
 - Learned – position embeddings learned during training.
 - Relative – encodes relative distance between tokens.

★ **INTERVIEW TIP:**

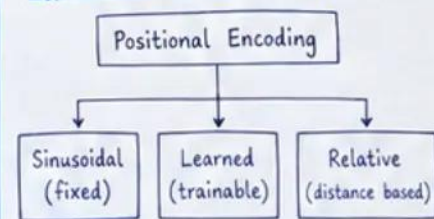
Be clear about why it is needed and how it helps the model.

Sinusoidal Positional Encoding (example)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Position	Dim 0 (sin)	Dim 1 (cos)
0	0.0000	1.0000
1	0.8415	0.5403
2	0.9093	-0.4161
3	0.1411	-0.9900

Types of Positional Encoding★ **IMPORTANT:**

- Positional encoding is added to input embeddings at the bottom of the model.
- Helps the model understand relative and absolute positions.

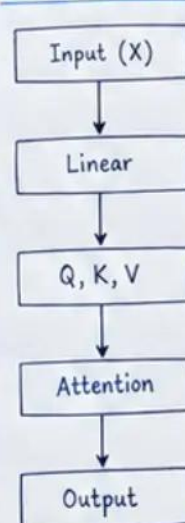
QUESTION 8 Explain the self-attention mechanism in detail. What are the roles of Query, Key and Value vectors? How does scaled dot-product attention work?

THEORETICAL SOLUTION:

- Self-attention allows each token to attend to all other tokens in the input sequence and compute a weighted representation.
- Query (Q) – what you are looking for
- Key (K) – what you are matching against
- Value (V) – the information to be aggregated
- Steps in scaled dot-product attention:
 - Compute similarity: $Q \cdot K^T$
 - Scale by $\sqrt{d_k}$
 - Apply softmax to get attention weights
 - Multiply with V to get output.
- It captures long-range dependencies and is highly parallelizable.

★ **FOLLOW-UP:**

What is the difference between self-attention and cross-attention? (Cross-attention uses encoder and decoder representations.)

Self-Attention Flow**Scaled Dot-Product Attention**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

C++ SOLUTION (simplified):

```

vector<vector<float>> scaledDotProduct(
    vector<vector<float>>& Q,
    vector<vector<float>>& K,
    vector<vector<float>>& V) {
    int n = Q.size();
    int d = Q[0].size();
    vector<vector<float>> scores(n, vector<float>(n));
    // Compute QK^T / sqrt(d)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            float dot = 0;
            for (int k = 0; k < d; k++) dot += Q[i][k] * K[j][k];
            scores[i][j] = dot / sqrt(d);
        }
    }
    // Apply softmax and multiply with V (omitted for brevity)
    return {};
}
  
```

TIME COMPLEXITY:

$$O(n^2 \cdot d)$$

SPACE COMPLEXITY:

$$O(n^2)$$

★ **IMPORTANT:**

- Scaling prevents large dot products.
- Multi-head attention uses multiple heads to attend to different subspaces.

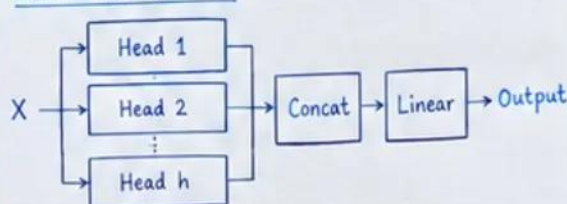
QUESTION 9 What is multi-head attention? Why is it better than single-head attention? What happens if we use only one head?

THEORETICAL SOLUTION:

- Multi-head attention uses multiple attention heads to attend to information from different representation subspaces at different positions.
- It allows the model to capture different types of relationships (syntactic, semantic, etc.) simultaneously.
- Single-head attention uses only one set of Q, K, V and may miss important patterns.

★ **INTERVIEW TIP:**

Be ready to explain with a diagram and a real-world analogy (e.g., different heads = different lenses).

Multi-Head Attention**Comparison**

Aspect	Single-Head	Multi-Head
Attention heads	1	Multiple
Representation	Single subspace	Multiple subspaces
Performance	Limited	Better
Complexity	Lower	Higher

Why better?

- Captures different types of relationships.
- More expressive representations.
- Improves performance on complex tasks.

★ **FOLLOW-UP:**

What happens if we use one head? → Model becomes less expressive and may miss important patterns.

★ **EDGE CASE:**

With very small data, multi-head may overfit.

3 INTERVIEW QUESTIONS

QUESTION 10 What is the purpose of positional encoding in Transformers?
What are the different types of positional encoding?

THEORETICAL SOLUTION:

- Transformers do not have recurrence or convolution, so they don't have a built-in way to understand the order of tokens.
- Positional encoding is added to input embeddings to give the model information about the position of each token in the sequence.
- It helps the model learn relative and absolute positions, which is important for tasks like translation, summarization, etc.

Types of Positional Encoding:

- Sinusoidal (Fixed)** — uses sine and cosine functions with different frequencies.
- Learned** — position embeddings are learned during training.
- Relative** — encodes relative distance between tokens (e.g., T5, DeBERTa).

Sinusoidal Positional Encoding (example)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Why needed?

- Gives sequence order information.
- Helps in capturing relative positions.
- Improves performance on sequence tasks.

★ INTERVIEW TIP:

Fixed (sinusoidal) encoding is better for generalization, while learned encoding can give better performance on specific tasks.

Comparison of Types

Type	How it works	Pros	Cons
Sinusoidal (Fixed)	Uses sine & cosine functions	Generalizes well, no extra params	May not be optimal for all tasks
Learned	Learned embeddings	Flexible, can learn patterns	Needs more data & parameters
Relative	Encodes relative distance	Good for long sequences	More complex implementation

★ IMPORTANT:

- Positional encoding is added to the input embeddings (not to the output).
- Different types are used depending on the task and model (e.g., BERT uses learned, T5 uses relative).

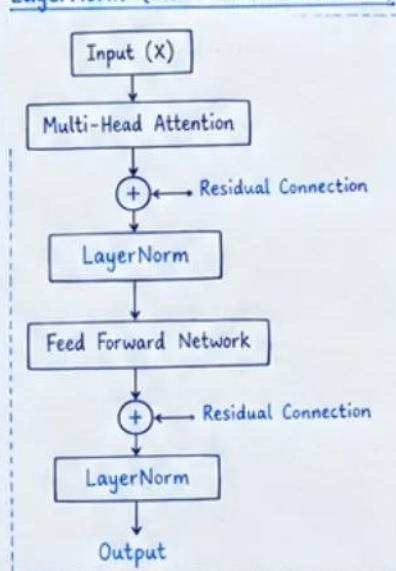
QUESTION 11 What is the role of Layer Normalization in Transformers?
How is it different from Batch Normalization?

THEORETICAL SOLUTION:

- Layer Normalization (LayerNorm) normalizes the activations across the feature dimension for each sample (independently).
- It stabilizes training, reduces internal covariate shift and allows for higher learning rates.
- It is applied after the attention and feed-forward layers in transformer blocks (with Residual connection).
- Unlike BatchNorm, it works better with variable sequence lengths and small batch sizes (common in NLP).

★ WHY USED?

- Faster and more stable training.
- Helps in better gradient flow.
- Works well with variable-length inputs.

LayerNorm (inside Transformer Block)**LayerNorm vs BatchNorm**

Aspect	LayerNorm	BatchNorm
Normalization axis	Across features (for each sample)	Across batch (for each feature)
Works well with	Variable batch sizes, sequence models	Large batch sizes, CNNs
Used in	Transformers, RNNs	CNNs, vision models
Stability	More stable for NLP	May be unstable for small batches

★ FOLLOW-UP:

- Why not use BatchNorm in Transformers?
→ Because sequence lengths vary and batch sizes can be small, making BatchNorm less effective.

QUESTION 12 How does the attention work in Transformers?
What are the different types of masks used?

THEORETICAL SOLUTION:

- Attention mask is used to prevent the model from attending to certain positions (e.g., padding tokens or future tokens).
- It is added to the attention scores before softmax, so the masked positions get very low probability.
- Different types of masks are used depending on the task.

Types of Masks:

- PADDING Mask** — masks padded tokens in variable-length sequences.
- Look-ahead Mask (Causal Mask)** — prevents attending to future tokens in decoder (used in language generation).
- COMBINED Mask** — often a combination of both (e.g., in seq2seq).

How it works?

- Compute attention scores: $\frac{QK^T}{\sqrt{d_k}}$
- Add mask (masked positions → $-\infty$)
- Apply softmax
- Get attention weights and output

Mask Example (Look-ahead)

Token	1	2	3	4
1	0	$-\infty$	$-\infty$	$-\infty$
2	0	0	$-\infty$	$-\infty$
3	0	0	0	$-\infty$
4	0	0	0	0

(0 = attend, $-\infty$ = masked)

Types of Masks (visual)**Padding Mask**

1	1	1	0	0
1	1	1	0	0
1	1	1	0	0

(0 = masked)

Look-ahead Mask

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0

(0 = masked)

★ IMPORTANT:

- The mask is added before softmax.
- Mask value is usually a large negative number (e.g., $-1e9$).
- Ensures correct attention behavior in both training and inference.

3 INTERVIEW QUESTIONS

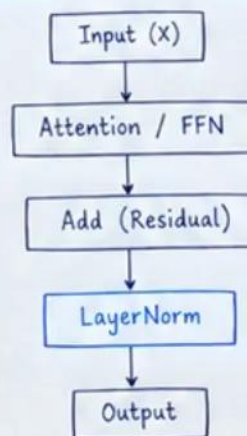
QUESTION 13 What is the role of Layer Normalization in Transformers?
How is it different from Batch Normalization?

THEORETICAL SOLUTION:

- **Layer Normalization (LN):** Normalizes the activations across the features of each individual sample (instead of across the batch).
- It helps in stabilizing training, reduces internal covariate shift and allows for higher learning rates. It is used in both encoder and decoder layers (after attention and feed-forward sublayers).
- **Batch Normalization (BN):** Normalizes across the batch dimension for each feature. It is more suitable for CNNs but not ideal for sequence models like Transformers because of variable sequence lengths and small batch sizes.

★ **INTERVIEW TIP:**

LN is preferred in Transformers because it is independent of batch size and works well with variable-length sequences.

Layer Normalization (inside a Transformer)**Comparison: LN vs BN**

Aspect	Layer Norm	Batch Norm
Normalization axis	Across features (for each sample)	Across batch (for each feature)
Depends on batch size	No	Yes
Works well with	Variable batch sizes, sequence models	Large batch sizes, CNNs
Used in	Transformers, RNNs	CNNs, vision models

★ **IMPORTANT:**

- LN normalizes per sample, BN per batch.
- LN is more stable for NLP tasks.

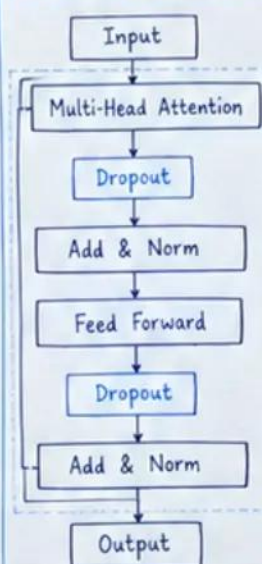
QUESTION 14 What is the purpose of Dropout in Transformers?
How does it work and what are its benefits?

THEORETICAL SOLUTION:

- **Dropout** is a regularization technique used to prevent overfitting by randomly setting a fraction of activations to zero during training.
- In Transformers, it is mainly used in:
 1. After the attention output (attention dropout).
 2. After the feed-forward layer (FFN dropout).
 3. In the embedding and positional encoding (embedding dropout).
- forces the model to not rely on specific neurons and improves generalization.

★ **INTERVIEW TIP:**

During inference, dropout is turned off and activations are scaled appropriately (no randomness).

Where Dropout is used**How it works?**

1. Choose a dropout rate p (e.g., 0.1).
2. Generate a random mask (Bernoulli).
3. Set activations to 0 where mask = 0.
4. Scale remaining activations by $1/(1-p)$.

Example ($p = 0.1$)

Input: [1.0, 2.0, 3.0, 4.0]
Mask: [1, 0, 1, 1]
After dropout: [1.11, 0, 3.33, 4.44]
(scaled by $1/(1-0.1) = 1.11$)

★ **BENEFITS:**

- Reduces overfitting.
- Improves generalization.
- Works well with large models.

Typical Dropout Rates

Component	Rate
Attention dropout	0.1
FFN dropout	0.1
Embedding dropout	0.1
Residual dropout	0.1

★ **FOLLOW-UP:**

- What happens if we use too high/low dropout rate?
- Why is dropout more useful than L2 regularization in some cases?

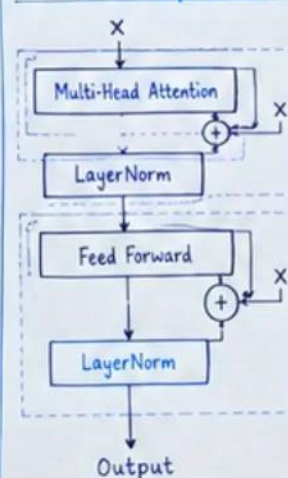
QUESTION 15 Explain the concept of Residual Connections in Transformers.
Why are they important and how do they help in training deep models?

THEORETICAL SOLUTION:

- **Residual Connection (skip connection)** adds the input of a layer to its output: $y = F(x) + x$.
- In Transformers, it is used around the Multi-Head Attention and Feed-Forward layers.
- It helps in:
 1. Solving the vanishing gradient problem.
 2. Allowing better information flow across layers.
 3. Enabling the training of deeper models.

★ **IMPORTANT:**

Residual connections are usually followed by Layer Normalization (in Pre-LN or Post-LN variants).

Transformer Layer with Residual Connection**Why are they important?**

1. Prevents gradient decay.
2. Stabilizes training.
3. Allows stacking of many layers.
4. Improves convergence speed.

★ **REMEMBER:**

Without residual connections, very deep Transformers become hard to train due to vanishing gradients.

Comparison: With vs Without Residual

Aspect	With Residual	Without Residual
Gradient flow	Better	Poor
Training deep	Easier	Difficult
Convergence	Faster	Slower
Performance	Higher	Lower

★ **FOLLOW-UP:**

- What happens if we remove residual connections?
- Why do we add LayerNorm after the residual connection?

3 INTERVIEW QUESTIONS

QUESTION 16 What is the purpose of positional encoding in Transformers?
What are the different types of positional encodings?

THEORETICAL SOLUTION:

- Transformers have no recurrence or convolution, so they do not have any built-in way to understand the order of tokens in a sequence.
- Positional encoding provides information about the position of each token, which helps the model learn the relative and absolute positions.
- It is added to the input embeddings to inject sequence order information.
- Without positional encoding, the model would treat all tokens as independent, which hurts tasks like translation, summarization, etc.

TYPES OF POSITIONAL ENCODING:

- Sinusoidal (Fixed)** — uses sine and cosine functions of different frequencies.
- Learned** — position embeddings are learned during training.
- Relative** — encodes relative distance between tokens (e.g., T5, DeBERTa).

★ **INTERVIEW TIP:**

Be clear about why it is required and how it helps the model.

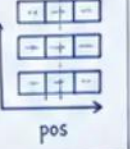
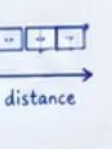
Sinusoidal Positional Encoding (example)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Position	Dim 0 (sin)	Dim 1 (cos)
0	0.0000	1.0000
1	0.8415	0.5403
2	0.9093	-0.4161
3	0.1411	-0.9900

Types of Positional Encoding

Sinusoidal (Fixed)	Learned (trainable)	Relative (distance based)
		
pos	pos	distance

Comparison:

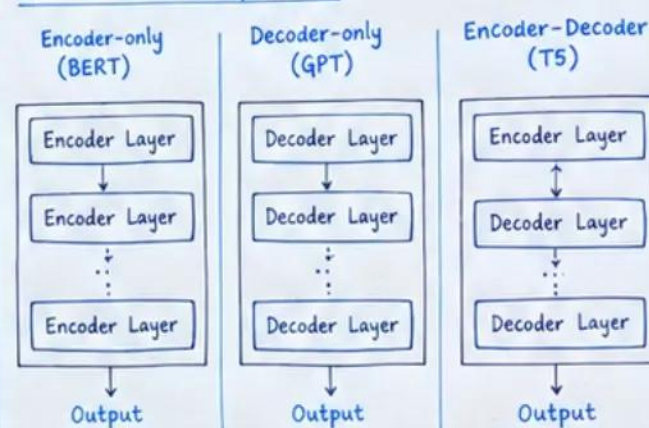
Feature	Sinusoidal	Learned	Relative
Nature	Fixed formula	Trainable	Encodes distance
Flexibility	Less flexible	More flexible	More flexible
Performance	Good	Often better in practice	Good

QUESTION 17 Explain the difference between encoder-only, aecoder-only and encoder-decoder Transformer architectures. Give one example of each.

THEORETICAL SOLUTION:

- Encoder-only:** Used for understanding tasks (e.g., classification, sentence embedding). It only has an encoder stack.
- Decoder-only:** Used for generation tasks (e.g., language modeling). It only has a decoder stack with causal masking.
- Encoder-Decoder:** Used for sequence-to-sequence tasks (e.g., translation, summarization). It has both encoder and decoder stacks with cross-attention.

Architecture	Used for	Example
Encoder-only	Understanding	BERT
Decoder-only	Generation	GPT
Encoder-Decoder	Seq2Seq	T5, BART

Architecture Comparison:★ **IMPORTANT:**

- Encoder-only → no generation ability.
- Decoder-only → needs causal mask.
- Encoder-decoder → needs cross-attention.

★ **FOLLOW-UP:**

Why does BERT use bidirectional attention while GPT uses causal masking?

QUESTION 18 What is the role of the feed-forward network (FFN) in Transformers?
How does it work and why is used?

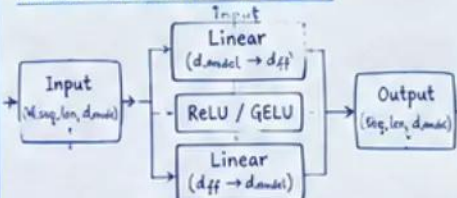
THEORETICAL SOLUTION:

- The FFN is a position-wise fully connected network applied to each token independently and identically.
- It consists of two linear layers with a non-linear activation (usually ReLU or GELU).
- It increases the model's capacity, helps in learning complex patterns and adds non-linearity after attention.
- FFN is applied after the attention layer with residual connection and layer normalization.

Common configuration:

$$FFN(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

(usually $d_{model} \rightarrow d_{ff} \rightarrow d_{model}$)

Position-wise Feed Forward Network**Why it is used?**

- Adds non-linearity to the model.
- Increases model capacity.
- Enables learning complex patterns.

★ **REMEMBER:**

FFN is applied separately to each position (not across tokens).

C++ SOLUTION (simplified):

```
struct FFN {
    Linear fc1, fc2;
    FFN(int d_model, int d_ff) {
        fc1 = Linear(d_model, d_ff);
        fc2 = Linear(d_ff, d_model);
    }
    Tensor forward(const Tensor& x) {
        x = fc1(x);
        x = gelu(x); // or relu(x)
        x = fc2(x);
        return x;
    }
};
```

TIME COMPLEXITY: $O(N \cdot d_{model} \cdot d_{ff})$

SPACE COMPLEXITY: $O(N \cdot d_{model} + d_{ff})$

★ **INTERVIEW TIP:**

Be ready to explain why the FFN is important and how it differs from the attention mechanism.

3 INTERVIEW QUESTIONS

QUESTION 19

What is the purpose of positional encoding in Transformers?
How does it work and what are the different types?

THEORETICAL SOLUTION:

- Positional encoding is used to inject information about the position of tokens in a sequence, since self-attention is position agnostic. Without it, the model would treat all tokens in a sequence as the same (order is lost).
- It helps the model learn relative and absolute positions, which is important for tasks like translation, summarization, etc.

TYPES OF POSITIONAL ENCODING:

- Sinusoidal (Fixed):** uses sine and cosine functions of different frequencies. Allows the model to attend by relative positions.
- Learned:** position embeddings are learned during training.
- Relative:** encodes relative distance between tokens (e.g., T5, DeBERTa).

★ INTERVIEW TIP:

Be clear about why it is required and how it helps the model.

Sinusoidal Positional Encoding (example)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

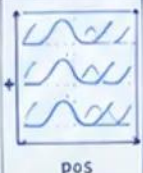
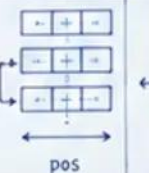
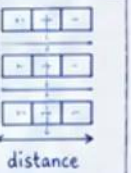
Why needed?

- Gives sequence order information.
- Helps in capturing relative positions.
- Improves performance on sequence tasks.

★ IMPORTANT:

Positional encoding is added to the input embeddings (not to the output).

Types of Positional Encoding

Sinusoidal (Fixed)	Learned (trainable)	Relative (distance based)
		

Comparison:

Feature	Sinusoidal	Learned	Relative
Nature	Fixed formula	Trainable	Encodes distance
Flexibility	Less flexible	More flexible	More flexible
Performance	Good	Often better in practice	Good

QUESTION 20

What is the role of self-attention in Transformers?
Why is it called 'self-attention' and how does it help the model?

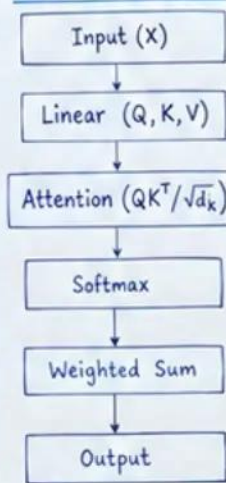
THEORETICAL SOLUTION:

- Self-attention allows each token to attend to all other tokens in the input sequence and compute a weighted representation.
- It is called self-attention because the attention is computed within the same sequence (no external memory).
- It helps the model capture long-range dependencies, handle variable-length inputs and learn contextual representations.

★ FOLLOW-UP:

How is self-attention different from RNNs / CNNs?
(RNNs are sequential, CNNs use local windows).

Self-Attention Flow



Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

• Why it works?

- Q (query): what you are looking for.
- K (key): what you can match with.
- V (value): information to aggregate.
- Softmax gives weights, then weighted sum.

Advantages:

- Captures long-range dependencies.
- More parallelizable than RNNs.
- Works well with variable-length inputs.

★ IMPORTANT:

Self-attention is the core of Transformer architecture.

QUESTION 21

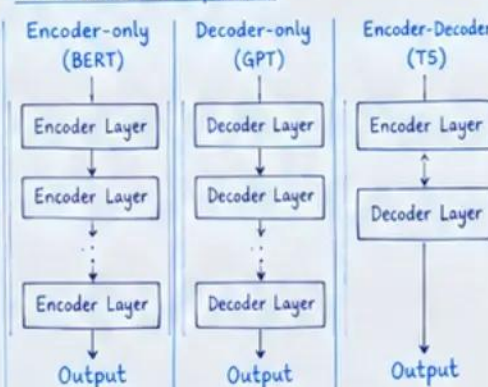
What is the difference between encoder-only, decoder-only and encoder-decoder Transformer architectures? Give one example of each.

THEORETICAL SOLUTION:

- Encoder-only:** used for understanding tasks (e.g., classification, sentence embedding). It only has an encoder stack.
- Decoder-only:** used for generation tasks (e.g., language modeling). It only has a decoder stack with causal masking.
- Encoder-Decoder:** used for sequence-to-sequence tasks (e.g., translation, summarization). It has both encoder and decoder stacks with cross-attention.

Architecture	Used for	Example
Encoder-only	Understanding	BERT
Decoder-only	Generation	GPT
Encoder-Decoder	Seq2Seq	T5, BART

Architecture Comparison



★ FOLLOW-UP:

Why does BERT use bidirectional attention while GPT uses causal masking?

Key Differences

Aspect	Encoder-only	Decoder-only	Encoder-Decoder
Attention	Bidirectional	Causal	Encoder: bidirectional Decoder: causal
Objective	Understanding	Generation	Seq2Seq
Example	BERT	GPT	T5, BART

★ IMPORTANT:

- Encoder-decoder models are more complex but better for tasks like translation.
- Encoder-only models are more efficient for classification/embedding tasks.

3 INTERVIEW QUESTIONS

QUESTION 25 What is the role of positional encoding in Transformers?
Why can't we just use the model without positional encoding?

THEORETICAL SOLUTION:

- Transformers are attention-based models and by themselves are permutation invariant. This means they cannot understand the order of tokens in a sequence.
- Positional encoding adds information about the relative or absolute position of tokens so that the model can use the sequence order while computing attention.
- It is added to the input embeddings (either fixed or learned) before feeding them into the first layer.
- Without positional encoding, different permutations of the same tokens would produce the same output, which is incorrect for tasks like translation or summarization.

★ **INTERVIEW TIP:**

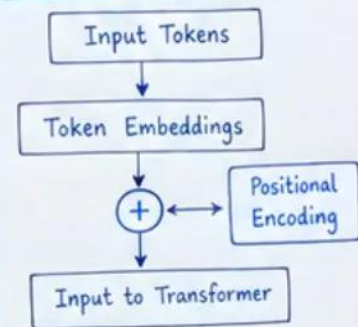
- Be clear that positional encoding is not about giving the model absolute position, but about providing sequence order information.

Types of Positional Encoding

Type	How it works
Sinusoidal (Fixed)	Uses sine & cosine functions of different frequencies.
Learned	Position embeddings are learned during training.
Relative	Encodes relative distance between tokens (e.g., T5, DeBERTa).

★ **Why it's needed?**

- Allows the model to capture sequence order.
- Helps in understanding relative positions.
- Improves performance on sequence tasks.

How it is added?★ **IMPORTANT:**

- Positional encoding is added to the input embeddings, not to the output.

QUESTION 26 What is the difference between masked self-attention and causal self-attention?
When is each used?

THEORETICAL SOLUTION:

- Masked self-attention allows each token to attend to all tokens in the input sequence (including future tokens). It is used in tasks like BERT (bidirectional language modelling).
- Causal self-attention (also called look-ahead masking) restricts each token to attend only to previous tokens (and itself), preventing it from seeing future tokens. It is used in autoregressive models like GPT for text generation.
- Both use a mask matrix to block certain positions in the attention scores before softmax.

Feature	Masked Self-Attention	Causal Self-Attention
Visible tokens	All tokens (bidirectional)	Past+current (unidirectional)
Mask type	None (or padding mask)	Look-ahead mask
Used in	BERT, RoBERTa	GPT, LLaMA, etc.

Mask Illustration**Masked Self-Attention**

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

(1 = can attend)

Causal Self Attention

1	0	0	0
1	1	0	0
1	1	1	0
1	1	1	1

(0 = masked)

★ **FOLLOW-UP:**

- Can we use causal attention in BERT?
- What are the other types of masks?
- How does masking affect training vs inference?

★ **IMPORTANT:**

- Masked attention is bidirectional.
- Causal attention is unidirectional (past only).
- Both are implemented by adding a large negative value ($-1e9$) to masked positions before softmax.

QUESTION 27 How does the Transformer approach 'used' in that transformers?
What are the existing issues and the coming things are suggestions?

THEORETICAL SOLUTION:

- Transformers require sequences to be of a fixed length, so padding is used for shorter sequences.
- A padding mask is created so that the model does not attend to the padding tokens.
- In addition, a special token (e.g., [CLS]) is often used for classification tasks to represent the whole sequence.
- For efficient training, batches are created using dynamic padding or bucketing to minimize the amount of padding.

★ **INTERVIEW TIP:**

- Remember the difference between padding mask (for padding tokens) and look-ahead mask (for causal attention).

Handling Variable Length Sequences

Seq 1:	10	25	6		(len = 3)
Seq 2:	8	14	7	6	(len = 4)
Seq 3:		15	9		(len = 2)

↓ Padding (to max length = 4)

Seq 1:	10	25	66	0
Seq 2:	8	14	7	6
Seq 3:	5	9	0	0

PADDING MASK (1 = keep, 0 = mask)

Seq 1:	1	1	1	0
Seq 2:	1	1	1	1
Seq 3:	1	1	0	0

Common Approaches

- Zero Padding — simple and widely used.
- Dynamic Padding — pad to longest sequence in the batch.
- Bucketing — group sequences with similar lengths.
- Packed Sequences (RNNs) — not used in Transformers.

★ **FOLLOW-UP:**

- How does padding affect training?
- What is the role of the attention mask in handling padding?

★ **IMPORTANT:**

- Padding tokens must be masked in both self-attention and cross-attention (if used).

3 INTERVIEW QUESTIONS

QUESTION 25 What is the purpose of positional encoding in Transformers?
How does it work and what are the different types of positional encoding?

THEORETICAL SOLUTION:

- Transformers are attention-based models and do not have any built-in notion of token order. Positional encoding is added to the input embeddings to give the model information about the position of each token in the sequence.
- It helps the model learn relative and absolute positions, which is important for tasks like translation, summarization, etc.
- Without positional encoding, different permutations of the same tokens would produce the same output.

Types of Positional Encoding:

- Sinusoidal (Fixed)** — uses sine and cosine functions with different frequencies.
- Learned** — position embeddings are learned during training.
- Relative** — encodes relative distance between tokens (e.g., T5, DeBERTa).

★ INTERVIEW TIP:

Be clear about why it is required and how it helps the model.

Sinusoidal Positional Encoding (example)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Why needed?

- Gives sequence order information.
- Helps in capturing relative positions.
- Improves performance on sequence tasks.

★ FOLLOW-UP:

Difference between absolute and relative positional encoding?

Comparison of Types

Type	How it works	Pros	Cons
Sinusoidal (Fixed)	Uses sine & cosine functions	Generalizes well, no extra params	May not be optimal for all tasks
Learned	Learned embeddings	Flexible, can learn patterns	Needs more data & parameters
Relative	Encodes relative distance	Good for long sequences	More complex implementation

★ IMPORTANT:

- Positional encoding is added to the input embeddings (not to the output).
- Different types are used depending on the task and model (e.g., BERT uses learned, T5 uses relative).

QUESTION 26 What is the difference between masked self-attention and causal self-attention?
When is each used?

THEORETICAL SOLUTION:

- Masked self-attention allows each token to attend to all tokens in the input sequence (including future tokens). It is used in tasks like BERT (bidirectional language modelling).
- Causal self-attention (also called look-ahead masking) restricts each token to attend only to previous tokens (not future tokens). It is used in autoregressive models like GPT (for text generation).
- Both use a mask matrix to block certain positions in the attention scores before softmax.

Feature	Masked Self-Attention	Causal Self-Attention
Visible tokens	All tokens (bidirectional)	Past + current (unidirectional)
Mask type	None (or padding mask)	Look-ahead mask
Used in	BERT, RoBERTa	GPT, LLAMA, etc.

Mask IllustrationMasked Self-Attention

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

(1 = can attend)

Causal Self-Attention

1	0	0	0
1	1	0	0
1	1	1	0
1	1	1	1

(0 = masked)

★ FOLLOW-UP:

- Can we use causal attention in BERT?
- What are the other types of masks?
- How does masking affect training vs inference?

★ IMPORTANT:

- Masked attention is bidirectional.
- Causal attention is unidirectional (past only).
- Both are implemented by adding a large negative value ($-1e9$) to masked positions before softmax.

★ REMEMBER:

The mask helps maintain the autoregressive property in generation tasks.

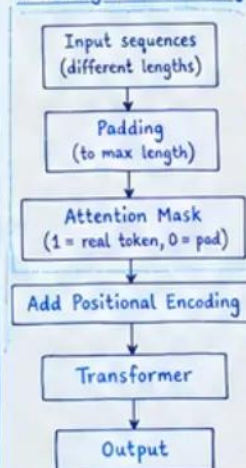
QUESTION 27 How does the Transformer handle variable length sequences?
What are the techniques used and why are they important?

THEORETICAL SOLUTION:

- Transformers require fixed-length input, so padding is used to make sequences of the same length in a batch.
- An attention mask is created to ignore the padded tokens so they do not affect the attention scores.
- Positional encoding is usually added before padding (or adjusted accordingly) to ensure correct position information.
- This allows efficient batch processing while preventing the model from attending to padding tokens.

★ COMMON MISTAKE:

Not masking padded tokens can lead to incorrect attention and poor performance.

Handling Variable Length SequencesExample

Batch (max length = 6)

Seq1: [101 23 45 0 0 0]
Seq2: [101 67 89 12 0 0]
Seq3: [101 34 78 56 43 0]

Attention Mask

Seq1: [1 1 1 0 0 0]
Seq2: [1 1 1 1 0 0]
Seq3: [1 1 1 1 1 0]

Why it is important?

- Prevents attention to padding tokens.
- Maintains correct positional information.
- Allows efficient batch training.

C++ SOLUTION (simplified):

```

vector<vector<int>> createMask(const vector<int>& len,
int maxlen) {
    vector<vector<int>> mask(len.size(),
vector<int>(maxlen, 0));
    for (int i = 0; i < len.size(); i++) {
        for (int j = 0; j < len[i]; j++) {
            mask[i][j] = 1;
        }
    }
    return mask;
}
  
```

TIME COMPLEXITY: $O(n \cdot m)$

SPACE COMPLEXITY: $O(n \cdot m)$

★ INTERVIEW TIP:

Remember to handle padding and masking together for correct attention in both training and inference.

3 INTERVIEW QUESTIONS

QUESTION 25 What is the purpose of positional encoding in Transformers?
How does it work and what are the different types?

THEORETICAL SOLUTION:

- Positional encoding is used to inject information about the position of tokens in a sequence, since self-attention itself is position agnostic.
- It helps the model understand the relative and absolute position of tokens in the input sequence.
- It is added to the input embeddings (or learned embeddings) before feeding them into the first layer.
- Without positional encoding, different permutations of the same tokens would produce the same output.

TYPES OF POSITIONAL ENCODING:

- Sinusoidal (Fixed)** — uses sine and cosine functions of different frequencies.
- Learned** — position embeddings are learned during training.
- Relative** — encodes relative distance between tokens (e.g., T5, DeBERTa).

★ **INTERVIEW TIP:**

Be clear about why it is required and how it helps the model.

Sinusoidal Positional Encoding (example)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Position	Dim 0 (sin)	Dim 1 (cos)
0	0.0000	1.0000
1	0.8415	0.5403
2	0.9093	-0.4161
3	0.1411	-0.9900

Comparison of Types

Type	How it works	Pros	Cons
Sinusoidal (Fixed)	Uses sine & cosine functions	Generalizes well, no extra params	May not be optimal for all tasks
Learned	Position embeddings are learned during training	Flexible, can learn patterns	Needs more parameters
Relative	Encodes relative distance	Good for long sequences	More complex implementation

★ **IMPORTANT:**

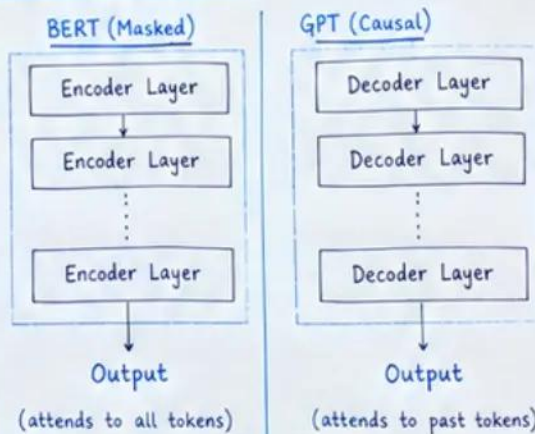
- Positional encoding is added to the input embeddings (not to the output).

QUESTION 26 What is the difference between masked self-attention and causal self-attention?
When is each used?

THEORETICAL SOLUTION:

- Masked self-attention allows each token to attend to all tokens in the input sequence (including future tokens). It is used in tasks like BERT (bidirectional language modelling).
- Causal self-attention (also called look-ahead masking) restricts each token to attend only to previous tokens (not future tokens). It is used in autoregressive models like GPT (text generation).
- Both use a mask matrix to block certain positions in the attention scores before softmax.

Feature	Masked Self-Attention	Causal Self-Attention
Visible tokens	All tokens (bidirectional)	Past + current (unidirectional)
Mask type	None (or padding mask)	Look-ahead mask
Used in	BERT, RoBERTa	GPT, LLaMA, etc.

Architecture Comparison:★ **FOLLOW-UP:**

- Can we use causal attention in BERT?
- What are the other types of masks?
- How does masking affect training vs inference?

★ **IMPORTANT:**

- Masked attention is bidirectional.
- Causal attention is unidirectional (past only).
- Both are implemented by adding a large negative value ($-1e9$) to masked positions before softmax.

★ **REMEMBER:**

- The mask helps maintain the autoregressive property in generation tasks.

QUESTION 27 How does the Transformer's feed-forward network (FFN) work?
What are the things coming in and what are they important?

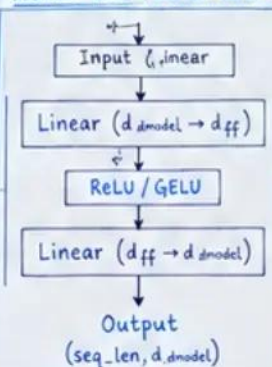
THEORETICAL SOLUTION:

- The FFN is a position-wise fully connected network applied to each token independently and identically.
- It consists of two linear layers with a non-linear activation (usually ReLU or GELU).
- It increases the model's capacity, helps in learning complex patterns and adds non-linearity after attention.
- Applied after the attention layer with residual connection and layer normalization.

Common Configuration:

$$FFN(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

(usually $d_{model} \rightarrow d_{ff} \rightarrow d_{model}$)

Position-wise Feed Forward Network**Why it is important?**

- Adds non-linearity to the model.
- Increases model capacity.
- Enables learning complex patterns.

★ **REMEMBER:**

FFN is applied separately to each position (not across tokens).

C++ SOLUTION (simplified):

```

struct FFN {
    Linear fc1, fc2;
    FFN(int d_model, int d_ff) {
        fc1 = Linear(d_model, d_ff);
        fc2 = Linear(d_ff, d_model);
    }
    Tensor forward(const Tensor& x) {
        x = fc1(x);
        x = gelu(x); // or relu(x)
        x = fc2(x);
        return x;
    }
};
  
```

TIME COMPLEXITY: $O(n \cdot d_{model} \cdot d_{ff})$

SPACE COMPLEXITY: $O(n \cdot d_{model} + d_{ff})$

★ **INTERVIEW TIP:**

Remember to handle padding and masking together for correct attention in both training and inference.

3 INTERVIEW QUESTIONS

QUESTION 31

What is the difference between causal self-attention and masked self-attention? When are they used?

THEORETICAL SOLUTION:

- Masked self-attention is a general term where a mask is applied to the attention scores to prevent attending to certain positions.
- Causal self-attention is a specific type of masked self-attention where the mask is structured to block future positions (i.e., only attend to current and past tokens).
- In practice, Causal self-attention is used in autoregressive models like GPT, while masked self-attention can also be used in tasks like BERT (bidirectional) with different mask patterns.

Key Differences:

Aspect	Masked Self-Attention	Causal Self-Attention
Mask type	General mask (any pattern)	Look-ahead mask (future blocked)
Attention scope	Depends on task	Only past + current
Used in	BERT, RoBERTa, etc.	GPT, LLaMA, etc.

Mask Illustration:

Masked Self-Attention (BERT)

1	1	0	1	1
1	1	1	0	1
0	1	1	1	1
1	0	1	1	1
1	1	1	1	0

(1 = attend, 0 = masked)

Causal Self-Attention (GPT)

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

(0 = masked)

★ INTERVIEW TIP:

Causal attention is a special case of masked attention with a triangular mask.

When used?

- Causal: Autoregressive generation (GPT, LLaMA).
- Masked: Bidirectional understanding (BERT).

★ IMPORTANT:

- Masking is done by adding $-1e9$ (or a large negative value) to the attention scores before softmax.

QUESTION 32

How does the Position-wise feed-forward network (FFN) work? Why is it used in Transformers?

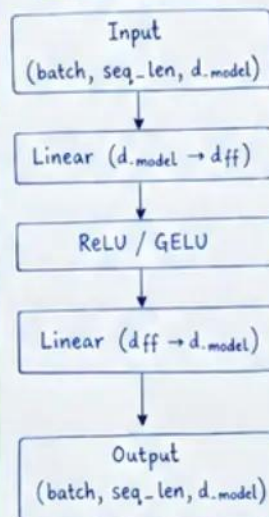
THEORETICAL SOLUTION:

- The FFN is a small neural network applied independently to each position (token) and identically across all positions.
- It consists of two linear layers with a non-linear activation (usually ReLU or GELU) in between.
- It helps the model learn complex, non-linear transformations of the token representations.
- The same FFN is used for every position, which allows the model to capture higher-level features without mixing information between tokens (that is handled by attention).

★ REMEMBER:

FFN is position-wise (no interaction between tokens) but shared across all positions.

FFN Architecture



Typical Dimensions

$d_{\text{model}} = 512$
 $d_{\text{ff}} = 2048$
 $(d_{\text{ff}} = 4 \times d_{\text{model}})$

Why is it used?

- Adds non-linearity to the model.
- Increases model capacity.
- Enables learning complex patterns.

C++ SOLUTION (simplified):

```

vector<vector<float>> ff(const vector<vector<float>>& x){
    int seq_len = x.size();
    int d_model = x[0].size();
    int d_ff = 4 * d_model;

    vector<vector<float>> out(seq_len, vector<float>(d_model));

    for (int i = 0; i < seq_len; i++) {
        for (int j = 0; j < d_model; j++) {
            float h1 = x[i][j] * W1[j];
            float h2 = max(0.0f, h1); // ReLU
            out[i][j] = h2 * W2[j];
        }
    }
    return out;
}
  
```

TIME COMPLEXITY: $O(n \cdot d_{\text{model}} \cdot d_{\text{ff}})$ SPACE COMPLEXITY: $O(n \cdot d_{\text{model}})$

★ INTERVIEW TIP:

The FFN is applied separately to each position (no cross-position interaction).

QUESTION 33

What are the advantages and disadvantages of the multi-head attention? Why is it better than single-head attention?

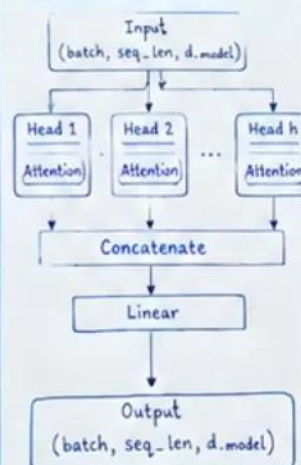
THEORETICAL SOLUTION:

- Multi-head attention uses multiple attention heads to attend to information from different representation subspaces at different positions.
- It allows the model to capture different types of relationships (e.g., syntactic, semantic, positional) in parallel.
- The Outputs of all heads are Concatenated and projected back to the model dimension.

Advantages:

- Captures multiple types of dependencies.
- Improves model performance and stability.
- Better representation learning.

Multi-Head Attention



Disadvantages:

- Higher computational cost.
- More memory usage.
- Slightly more complex implementation.

Comparison (Single vs Multi-Head)

Aspect	Single-Head	Multi-Head
Attention heads	1	> 1
Representation	Limited	Multiple subspaces
Performance	Lower	Better
Computation	Lower	Higher

★ FOLLOW-UP:

- How many heads are typically used? (e.g., 8, 16)
- What happens if we use too many heads?

★ IMPORTANT:

Multi-head attention is more expressive and better at capturing complex relationships than single-head.

Edge Case:

If the number of heads does not divide d_{model} , we use padding or adjust head size.

3 INTERVIEW QUESTIONS

QUESTION 34 What is the difference between self-attention and cross-attention? When is cross-attention used?

THEORETICAL SOLUTION:

- Self-attention allows each token to attend to all tokens in the same sequence. It is used in encoder (or decoder) self-attention.
- Cross-attention allows tokens from one sequence to attend to tokens from another sequence. It is used in the decoder to attend to encoder outputs (e.g., in encoder-decoder models).
- Self-attention uses Q, K, V from the same input, while Cross-attention uses Q from the target (decoder) and K, V from the source (encoder).

★ INTERVIEW TIP:

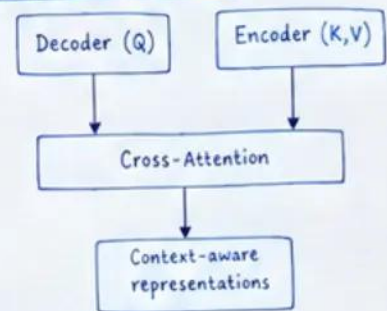
Cross-attention is essential in sequence-to-sequence tasks like translation, summarization and image captioning.

Comparison

	Self-Attention	Cross-Attention
Source of Q,K,V	Same sequence	Different sequences
Used in	Encoder / Decoder self-attention	Decoder (encoder-decoder)
Purpose	Capture intra-sequence context	Attend to external context

When used?

- Machine translation
- Summarization
- Image captioning
- Question answering

Cross-Attention (Decoder)★ IMPORTANT:

- Self-attention captures "what to focus on in the same sequence".
- Cross-attention captures "what to focus on from another sequence".

QUESTION 35 What is the difference in transformers? Why is it used instead of batch normalization?

THEORETICAL SOLUTION:

- Layer normalization (LN) normalizes the activations across the feature dimension for each sample (not across the batch).
- It helps stabilize training, especially with variable sequence lengths and small batch sizes.
- It is used instead of batch normalization because BN depends on batch statistics, which can be less effective in NLP tasks.
- LN works for both training and inference in the same way.

★ REMEMBER:

LN is applied after the residual connection (Add & Norm) in the Transformer architecture.

How it works?

- For each token (sample), compute mean and variance across features.
- Normalize using:

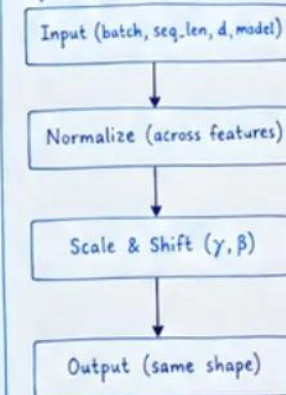
$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- Scale and shift using learnable parameters γ and β :

$$y = \gamma \hat{x} + \beta$$

Where:

- μ, σ^2 → mean and variance (per sample)
- γ, β → learnable parameters
- ϵ → small value for numerical stability

Layer NormalizationWhy used?

- Independent of batch size.
- Works well with variable sequence lengths.
- More stable in NLP tasks.
- Better suited for autoregressive models.

★ FOLLOW-UP:

- How does LN differ from RMSNorm?
- Where is LN applied in the Transformer?
- What are its advantages and limitations?

QUESTION 36 How does the positional encoding work in transformers? What are the different types of positional encoding?

THEORETICAL SOLUTION:

- Transformers are permutation-invariant, so positional information is added to input embeddings.
- Positional encoding provides information about the position of tokens in the sequence.
- It is added to the input embeddings at the bottom of the model (encoder and decoder).
- Different types include sinusoidal, learned and relative positional encoding.

★ INTERVIEW TIP:

Sinusoidal encoding is commonly used, while learned encoding can give better performance in some tasks.

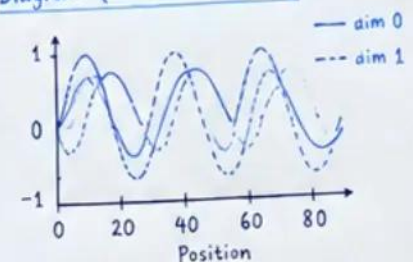
Types of Positional Encoding

Type	How it works	Pros	Cons
Sinusoidal (Fixed)	Uses sine & cosine functions of different frequencies.	Generalizes to longer sequences.	May not be optimal for all tasks.
Learned	Position embeddings are learned during training.	Flexible, can learn patterns.	Needs more parameters.
Relative	Encodes relative distance between tokens.	Good for long sequences.	More complex implementation.

Example (Sinusoidal):

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Diagram (Sinusoidal Encoding)★ IMPORTANT:

- Positional encoding is added to input embeddings, not to the output.
- Relative encoding is useful for tasks like translation with long sequences.
- Learned encoding often gives better results in practice.

3 INTERVIEW QUESTIONS

QUESTION 37

How does the Transformer handle long sequences?
What are the techniques used to improve its efficiency?

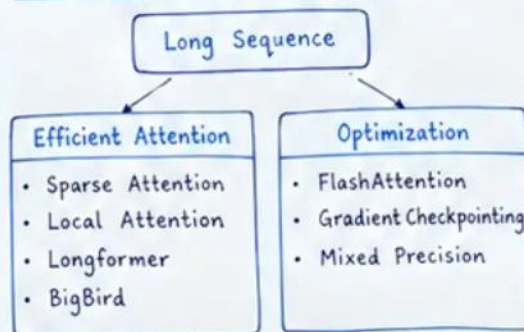
THEORETICAL SOLUTION:

- The ORIGINAL Transformer has a quadratic complexity $O(n^2)$ due to self-attention, which becomes expensive for long sequences.
- Several techniques are used to handle long sequences efficiently:
 1. Sparse Attention — attends only to a subset of tokens.
 2. Local Attention — restricts attention to a fixed window.
 3. Hierarchical / Longformer / BigBird — use different patterns to reduce computation.
 4. Flash Attention — optimizes memory and computation on GPU.
 5. Gradient checkpointing — reduces memory usage during training.
- These techniques reduce memory and computation while maintaining performance.

★ INTERVIEW TIP:

Know at least 2-3 methods and when each one is preferred (e.g., Longformer for long documents).

Techniques for Long Sequences



When used?

- Documents (NLP), genomic data (bioinformatics).
- Large videos / multi-modal inputs.
- When sequence length $> 2k - 4k$ tokens.

★ IMPORTANT:

- Sparse / Local attention reduces complexity to $O(nk)$ where $k \ll n$.
- FlashAttention improves GPU memory bandwidth.

★ FOLLOW-UP:

- What is the maximum context length of GPT-3/4?
- How does RoPE help with long sequences?
- Why is linear attention less commonly used?

QUESTION 38

What is the Role of Positional encoding in Transformers?
How do different types compare?

THEORETICAL SOLUTION:

- Positional encoding provides information about the position of tokens in the sequence, since attention is position-agnostic.
- Without it, the model cannot distinguish between different word orders (e.g., "dog bites man" vs "man bites dog").
- Types of Positional encoding:
 1. Sinusoidal (fixed) — uses sine and cosine functions.
 2. Learned — position embeddings are learned during training.
 3. Relative — encodes relative distance between tokens.
- Each type has trade-offs in terms of flexibility, generalization and performance.

★ REMEMBER:

Positional encoding is added to token embeddings, not to the output.

Comparison of Positional Encoding

Type	How it works	Pros	Cons
Sinusoidal (Fixed)	Sine & cosine functions with different frequencies.	Generalizes well, works for longer sequences.	May not be optimal for all tasks.
Learned	Learnable embeddings during training.	Flexible, can learn patterns.	Needs more parameters.
Relative	Encodes distance between tokens (e.g., T5, DeBERTa).	Good for long sequences.	More complex implementation.

Sinusoidal Positional Encoding (example)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

★ IMPORTANT:

- Learned embeddings are better for small datasets.
- Relative encoding works well for tasks like translation and summarization.

★ FOLLOW-UP:

- Why does RoPE work well in LLMs?
- Can we use absolute encoding with BERT?
- What happens if we remove positional encoding?

QUESTION 39

How does the Transformer handle variable-length sequences?
What techniques are used for correct batching and masking?

THEORETICAL SOLUTION:

- Real-world data has variable sequence lengths, so padding is used to make batches of equal size.
- Attention masks are used to prevent the model from attending to padded tokens.
- Two types of masks:
 1. Padding mask — masks out padding positions.
 2. Causal mask — ensures autoregressive behavior (in decoder).
- Masking is added before softmax so that ignored positions get very low attention weights.

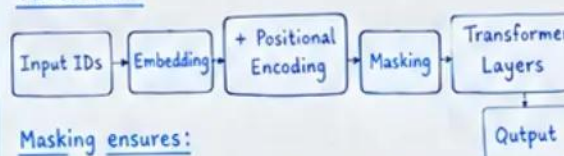
★ INTERVIEW TIP:

Remember to handle padding and masking correctly in both training and inference.

Input with Padding + Masks

Tokens	I	love	NLP	<pad>	<pad>
Token IDs	101	202	303	0	0
Padding Mask	1	1	1	0	0
Causal Mask	1	1	1	1	1

Model Flow



Masking ensures:

- Padded tokens don't affect attention.
- Decoder doesn't attend to future tokens.

★ IMPORTANT:

- Use additive mask (-1e9) before softmax for masked positions.
- In PyTorch, use `attention_mask` and `key_padding_mask`.

★ FOLLOW-UP:

- How is padding handled in variable-length batches?
- What is the difference between padding mask and causal mask?
- Can we avoid padding altogether?

★ REMEMBER:

Masking is essential for correct training and generation.

3 INTERVIEW QUESTIONS

QUESTION 40

What are the advantages and disadvantages of using Transformers in sequence modeling? When might other models be better?

THEORETICAL SOLUTION:

★ Advantages:

- Captures long-range dependencies effectively (via self-attention).
- Highly parallelizable (faster training).
- Better performance on large-scale data.
- Flexible — works for various tasks (NLP, vision, etc.).

★ Disadvantages:

- Higher computational and memory cost ($O(n^2)$ attention).
- Requires large amounts of data for good performance.
- Positional encoding is needed (unlike RNNs).
- Can be overkill for short sequences.

★ INTERVIEW TIP:

Transformers are great for long sequences and large data. For short sequences or low-resource settings, RNNs or CNNs may be better.

Comparison with Other Models

Aspect	Transformer	RNN/LSTM	CNN
Long-range dependencies	Excellent	Hard to learn	Limited
Parallelization	Yes	No	Yes
Training speed	Fast	Slow	Fast
Memory usage	High ($O(n^2)$)	Low	Moderate
Best use case	Large data, long sequences	Short sequences, small data	Local patterns (e.g., vision)

When others are better?

- RNN/LSTM: very short sequences, streaming data, low compute.
- CNN: when local features are more important (e.g., images, speech).
- Also useful when training data is limited.

★ REMEMBER:

Transformers trade memory for performance.

QUESTION 41

How does the Prefixtuning / prompt tuning work in Transformers? What are its used FORM-ENS?

THEORETICAL SOLUTION:

- Prefix tuning adds trainable "virtual tokens" (prefix) to the input sequence. Only the prefix parameters are trained, while the original model weights are kept frozen.
- Prompt tuning is similar but uses a small set of learnable tokens as prompts at large input layer.
- These learned tokens guide the model too produce the desired output without updating the entire model.

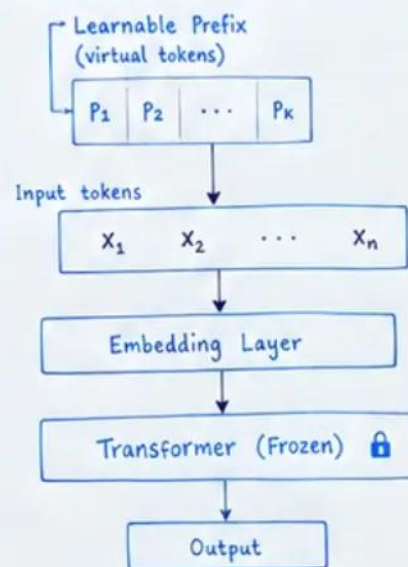
How it works (Prefix Tuning):

1. Learn a small set of prefix embeddings.
2. Concatenate them with the input embeddings.
3. Pass through the fixed Transformer.
4. Only the prefix parameters are updated during training.

★ INTERVIEW TIP:

Used when compute/MEMORY is limited or when quick adaptation is needed for multiple tasks.

Prefix Tuning (Simplified)



★ Advantages:

- Much fewer trainable parameters.
- Lower memory and compute cost.
- Faster training.
- Works well for multi-task learning and large models.

★ Disadvantages:

- Slightly lower performance than full fine-tuning (in some cases).
- Requires careful tuning of prefix length / size.
- Not always effective for very complex tasks.

★ FOLLOW-UP:

- How do you choose the prefix length?
- Can it be combined with other methods (e.g., LoRA)?

QUESTION 42

How does the Transformer handle variable-length sequences? What are the common techniques used and their trade-offs?

THEORETICAL SOLUTION:

- Transformers require fixed-length input, so variable-length sequences are handled using padding and attention masks.
- Padding ensures all sequences in a batch have the same length.
- Attention masks prevent the model from attending to padding tokens (using $-\infty$ or a large negative value).
- Positional encoding is applied only to real tokens (not padding).

★ COMMON MISTAKE:

Not using proper masking can lead to attention over padding tokens, which affects performance.

Example

Input:	[101 205 307 0 0]
Mask:	[1 1 1 0 0]

(0 = padding token)

Techniques

Method	Pros	Cons
Padding + Mask	Simple, effective	Wastes compute on padding.
Dynamic Batching	Better efficiency	More complex implementation.
Packed Sequences	No padding needed	Not supported in all frameworks.

★ Trade-offs:

- Padding + Mask: easy to implement, widely used, but less efficient.
- Dynamic batching: improves efficiency, but adds complexity.
- Packed sequences: avoids padding, but harder to implement and use.

★ IMPORTANT:

Masking is essential for correct attention and loss computation.

★ FOLLOW-UP:

How does variable length affect training and inference speed?

3 INTERVIEW QUESTIONS

QUESTION 43

What are the main differences between BERT and GPT?
When would you use each of them?

THEORETICAL SOLUTION:

- BERT (ENCODER-ONLY) uses bidirectional self-attention, meaning it looks at both LEFT and RIGHT context. It is trained with a masked language modelling (MLM) objective.
- GPT (DECODER-ONLY) uses causal self-attention, which only looks at past tokens. It is trained with next token prediction.
- BERT is better for UNDERSTANDING tasks (classification, NER, QA) while GPT is better for generation tasks (chat, summarization, code generation).

★ INTERVIEW TIP:

- BERT UNDERSTANDS context, GPT generates text.
The choice depends on the task.

Comparison Table:

Feature	BERT	GPT
Architecture	Encoder-only	Decoder-only
Attention	Bidirectional	Causal (unidirectional)
Training Objective	Masked LM	Next token prediction
Best for	Understanding (Cls, NER, QA)	Generation (Chat, Summarization, Code, etc.)
Context usage	Both left & right	Only left (past)

★ FOLLOW-UP:

- Can we use BERT for generation?
- Can we use GPT for classification?
- What are the performance trade-offs?

★ IMPORTANT:

- BERT = understands the input.
- GPT = generates the output.
- Both are based on Transformer architecture.

★ REMEMBER:

BERT uses MLM,
GPT uses causal LM.

QUESTION 44

How does POSITIONAL ENCODING WORK?
Why is it REQUIRED in TRANSFORMERS?

THEORETICAL SOLUTION:

- TRANSFORMERS HAVE NO RECURRENT LAYERS, so they don't inherently know the order of tokens.
- POSITIONAL ENCODING adds information about token positions using sine and cosine functions of different frequencies.
- It is added to the INPUT EMBEDDINGS so the model can use position information while attending to tokens.
- Each position gets a UNIQUE VECTOR, so the model can learn relative and absolute positions.

★ INTERVIEW TIP:

- Without positional encoding, the model cannot distinguish between different word orders (e.g., "dog bites man" vs "man bites dog").

Positional ENCODING FORMULA:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Where:

- pos = position of the token
- i = dimension
- d_{model} = embedding size

Example (d_{model} = 6):

Pos	Dim 0	Dim 1	Dim 2	Dim 3	Dim 4	Dim 5
0	0.000	1.000	0.000	1.000	0.000	1.000
1	0.841	0.540	0.010	0.999	0.001	1.000
2	0.909	-0.416	0.020	0.999	0.002	1.000

★ REQUIRED?

- Gives position information to the model.
- Helps the model learn order and relative distance.
- Enables better generalization to longer sequences.

★ FOLLOW-UP:

- Can we learn positional embeddings instead?
- What are the pros and cons of both approaches?
- How does it work with relative positional encoding?

QUESTION 45

How does BEAM SEARCH WORK in TRANSFORMERS?
What are its ADVANTAGES and LIMITATIONS?

THEORETICAL SOLUTION:

- Beam search is used during text generation to keep multiple likely candidates (beams) instead of only the best one.
- At each step, it expands the top-k candidates and keeps the best k based on probability (log probability).
- It helps improve the quality of generated text compared to greedy decoding.
- It balances between exploration and efficiency.

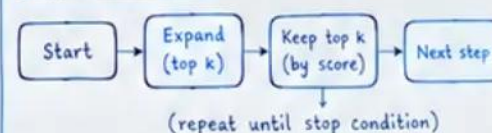
★ REMEMBER:

- Greedy decoding keeps only 1 candidate.
Beam search keeps top-k candidates.

How it works (steps):

- Start with initial token (e.g., <bos>)
- Expand each beam with top k next tokens.
- Keep top k candidates based on score.
- Repeat until <eos> or max length.

Diagram (k = 2):



★ INTERVIEW TIP:

- Use log probabilities to avoid underflow when multiplying many probabilities.

Advantages & Limitations:

Advantages	Limitations
Better quality than greedy.	More computationally expensive.
Reduces risk of poor choices.	May still miss the best output.
Works well for longer sequences.	Requires tuning of beam size (k).

★ FOLLOW-UP:

- How to choose the value of k?
- Can we use nucleus/top-k sampling instead?
- How does beam search affect latency?

★ IMPORTANT:

- Beam search is widely used in translation, summarization, and text generation tasks.